

Hash 与直接定位

用槽位和冲突管理换取接近直接访问

408 · 数据结构 Topic DS-10

母问题：映射不是定位，冲突怎样成为状态的一部分？

散列的生成链是

Key → Hash Mapping → Collision → Resolution → Load State → Expected Cost.

哈希函数只给出候选槽位，不保证唯一；开放定址沿探测序列寻找空位，链地址把同槽 key 放入桶链。查找失败必须按同一探测规则走到“从未使用”或完整探测，删除不能简单清空开放定址槽位，否则会截断后续 key 的探测链。

本册定位与 Owner 边界

- **Owns**：散列函数、装填因子、开放定址/链地址、删除标记、重散列与成功/失败查找语义。
- **Uses**：Atlas Foundation 与 DS-B02 workload 比较；精确等值查询优先，不承诺范围顺序。
- **Stops**：有序索引归 DS09；具体工程哈希表的并发、缓存和安全策略只作 Extension。

目录

1	状态与冲突策略	2
1.1	哈希函数先看取值域与分布	2
1.2	链地址与开放定址的两种成本	2
1.3	装填因子、删除与重散列	2
2	成本与边界	2
3	Hash 作为辅助索引：把全局扫描改成局部证据	3
3.1	配对、分组与连续段	3
3.2	对象到位置的反向索引	3
4	代码与测试证据	3
5	做题控制协议	4

1 状态与冲突策略

开放定址槽位至少有 Empty、Occupied、Deleted 三态；链地址槽位可用容器表示。装填因子 $\alpha = n/m$ 越高，冲突和探测长度通常越大；超过阈值应扩容并重新插入，不能直接把旧下标复制到新表。删除标记仍占探测路径，重散列时可被清除。

为什么删除后不能变 Empty

若 $h(a) = h(b)$ ，先插入 a 再插入 b ， b 可能探测到下一个槽。删除 a 后若把槽清空，查找 b 看到第一个 Empty 就会错误失败；Deleted 表示“继续探测，但此处可复用”。

1.1 哈希函数先看取值域与分布

直接定址 $H(k) = k$ 只适合 key 值域小且连续的场景；常见除留余数法写成 $H(k) = k \bmod m$ ，表长 m 通常选质数以减少周期性碰撞。折叠、平方取中等方法适合 key 的位分布不同的场景，但任何函数都只能给出候选地址，不能证明无冲突。分析失败查找的 ASL 时，入口数由 H 的实际值域决定，而不是看到容量 m 就直接除以 m 。

1.2 链地址与开放定址的两种成本

链地址把同一槽位的记录挂在桶链上，装填因子可以超过 1；查找先计算桶，再在链内比较。均匀散列下，成功查找的平均链长约为 $1 + \alpha/2$ ，失败查找约为 α （具体常数随“插入前/插入后计数”口径变化，题目应按给定定义计算）。删除可直接摘除链节点，但要处理头节点和重复 key。

开放定址要求 $\alpha < 1$ ，所有记录都在表内，冲突后按探测序列

$$H_i(k) = (H(k) + i) \bmod m$$

或二次探测、双重散列继续找槽。线性探测实现简单但会产生主聚集；二次探测减轻主聚集，却要检查表长与步长是否覆盖所有槽；双重散列的第二步长必须与 m 互素，否则可能只访问部分槽位。探测序列一旦绕回起点仍未命中，就应报告满表/失败而不是继续循环。

1.3 装填因子、删除与重散列

插入时应区分“第一次 Deleted 槽位”和“真正 Empty 槽位”：可以记住第一个 Deleted 位置，但仍需继续探测，确认 key 不在后再复用它。扩容后每个 key 的 $H(k) \bmod m'$ 都可能改变，必须按新表长重新插入，不能把旧数组下标整体复制。缩容、长期 tombstone 堆积和链地址桶过长都是重散列触发条件；扩容阈值是工程策略，题目若未给定不能擅自当成固定常数。

哈希题的失败证明

成功查找：按同一探测序列遇到目标。失败查找：开放定址遇到从未使用的 Empty，或已经完整探测一周；链地址扫描完桶链。Deleted 只能表示“此处曾经有记录”，不能作为开放定址的失败终点。把三种状态和终止条件写在草稿上，才能避免删除后误报不存在或满表死循环。

2 成本与边界

均匀散列、合理装填因子和独立探测假设下，开放定址成功/失败查找期望接近常数；最坏可退化到 $O(m)$ 。哈希不保序，因此范围查询、前驱后继和有序遍历不能由平均 $O(1)$ 推出。

3 Hash 作为辅助索引：把全局扫描改成局部证据

Hash 的算法价值常常不是“单独存一张表”，而是为扫描过程维护已见集合、频次或对象到位置的反向索引。正确性仍来自外层算法不变量，期望复杂度则额外依赖哈希分布与装填控制。

3.1 配对、分组与连续段

两数之和的一遍扫描维护“此前已经出现的值 $\rightarrow$ 位置”。处理 x 时先查询目标补数 $t - x$ ，命中后得到两个不同位置；再插入当前 x ，从而不会在只出现一次时重复使用同一元素。若需要所有配对，重复值、输出去重和位置组合会改变接口，不能沿用“找到即返回”的状态。

异位词分组需要把等价对象压成同一规范 key：把每个字符串排序后作为 key，每项成本 $O(L \log L)$ ；固定小字符集时用频次数组作 key，可降为 $O(L + |\Sigma|)$ ，但字符集和整数编码必须固定。Hash 只负责把相同 key 聚到一起，“规范化是否保持且仅保持目标等价关系”才是分组正确性的核心。

最长连续整数段先把所有不同值放入集合，只从不存在 $x - 1$ 的值 x 开始向右扩展。这样每个值至多被一条真正的段扫描访问一次，均匀散列假设下期望 $O(n)$ ；若从每个值都向两侧扩展，会对长段反复扫描而退化。重复输入应在集合化阶段消除，极值处计算 $x - 1$ 或 $x + 1$ 还需处理整数溢出。

3.2 对象到位置的反向索引

当数组支持尾部 $O(1)$ 插入/删除时，Hash 可维护 $\text{value} \rightarrow \text{index}$ ：删除任意值时把数组末元素搬到空位，并同步更新其索引。这是 RandomizedSet、频率桶和缓存结构常用接口，但组合结构还必须维护数组、链表、频次桶等跨索引一致性；完整的 workload、失败原子性与 LRU/LFU 推导由 DS-I01 Own，本册只提供反向定位机制。

期望线性不是无条件线性

上述应用把每次 Hash 操作按期望 $O(1)$ 计；对抗输入、退化哈希函数、重散列峰值或必须给出最坏界时，应改用有序树、排序或受控哈希策略，并重新报告成本向量。

4 代码与测试证据

完整实现位于 `code/ds10_hash_table.hpp`，测试位于 `code/ds10_hash_table_test.cpp`。

```

1 public:
2     explicit LinearHashTable(std::size_t capacity = 8) : slots_(capacity ?
        capacity : 1) {}
3     bool insert(int key) { if ((size_ + 1) * 10 >= slots_.size() * 7) rehash(
        slots_.size() * 2); return place(key); }
4     bool contains(int key) const { return locate(key).has_value(); }
5     bool erase(int key) { auto index = locate(key); if (!index) return false;
        slots_[*index] = Slot::deleted(); --size_; return true; }
6     std::size_t size() const { return size_; }
7
8 private:
9     struct Slot { std::optional<int> value; bool tombstone = false; static Slot
        deleted() { return Slot{std::nullopt, true}; } };
10    std::vector<Slot> slots_; std::size_t size_ = 0;
11    std::size_t hash(int key) const { return static_cast<std::size_t>(key >= 0 ?

```

```

    key : -static_cast<long long>(key)) % slots_.size(); }
12  bool place(int key) { std::size_t first_deleted = slots_.size(); for (std::
    size_t step = 0; step < slots_.size(); ++step) { auto i = (hash(key) + step
    ) % slots_.size(); if (slots_[i].value && *slots_[i].value == key) return
    false; if (!slots_[i].value) { if (first_deleted != slots_.size()) i =
    first_deleted; slots_[i].value = key; slots_[i].tombstone = false; ++size_;
    return true; } if (slots_[i].tombstone && first_deleted == slots_.size())
    first_deleted = i; } return false; }
13  std::optional<std::size_t> locate(int key) const { for (std::size_t step = 0;
    step < slots_.size(); ++step) { auto i = (hash(key) + step) % slots_.size
    (); if (!slots_[i].value && !slots_[i].tombstone) return std::nullopt; if (
    slots_[i].value && *slots_[i].value == key) return i; } return std::nullopt;
    }
14  void rehash(std::size_t capacity) { auto old = std::move(slots_); slots_.
    assign(capacity, Slot{}); size_ = 0; for (const auto& slot : old) if (slot.
    value) place(*slot.value); }
15 };
16 } // namespace ipara::ds10
17
18 #endif

```

代码清单 1: 线性探测、删除标记与重散列

```

1  assert(table.contains(5)); assert(table.erase(1)); assert(!table.contains(1)
    );
2  assert(table.contains(5));
3  for (int value = 10; value < 20; ++value) assert(table.insert(value));
4  assert(table.size() == 11);
5  }

```

代码清单 2: 冲突、删除后查找与满表边界测试

```

1  clang++ -std=c++17 -Wall -Wextra -Werror -pedantic code/ds10_hash_table_test.
    cpp -o /tmp/ds10_test
2  /tmp/ds10_test

```

5 做题控制协议

先问查询是否精确等值，再写 $h(k)$ 、冲突处理、装填因子和失败终止条件。开放定址题画探测序列，删除保留 tombstone；扩容题重新计算每个 key 的槽位。若题目要求范围或排序，转交 DS09/DS11。

压缩复原

五问：key 映射到哪里？冲突后走哪条路径？槽位有几种状态？失败何时停止？装填变大如何修复？